

Spring & Spring Boot — Complete Beginner Guide

This topic is the **starting point of Spring Boot**. The goal is not to learn advanced Spring yet, but to understand **why Spring exists, what Spring Boot adds, and what the generated application is actually doing**.

1. What Is a Framework?

A **framework** is a reusable structure that provides common functionality and conventions so developers don't have to build everything from scratch.

Without a framework

Suppose you build a Java web application manually. You may need to handle:

- HTTP requests
- HTTP responses
- Object creation
- Dependency management
- Configuration
- Server setup
- Routing
- Error handling

You would write a lot of infrastructure code yourself.

With a framework

The framework provides much of this infrastructure.

You concentrate more on:

```
Your Business Logic
    ↓
Framework
    ↓
Infrastructure
```

Simple example

Without a framework:

```
public class Main {  
    public static void main(String[] args) {  
        UserService service = new UserService();  
        UserRepository repository = new UserRepository();  
  
        service.setRepository(repository);  
  
        service.createUser();  
    }  
}
```

You manually create and connect objects.

With Spring, Spring can manage those objects and their relationships for you.

2. What Is Spring?

Spring is a Java framework used to build applications by providing infrastructure such as dependency management, object management, web support, configuration, and many other features.

The most important beginner concept is:

Spring manages objects for you.

These Spring-managed objects are called **beans**.

For example:

```
public class UserService {  
  
    public void createUser() {  
        System.out.println("User created");  
    }  
}
```

Normally:

```
UserService service = new UserService();
```

Spring can create and manage the UserService object instead.

3. Why Does Spring Exist?

Imagine a small application:

```
Main
└─ UserService
   └─ UserRepository
```

You can manually create everything:

```
UserRepository repository = new UserRepository();
UserService service = new UserService(repository);
```

That's fine.

But imagine a large application:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database

Service
  ↓
EmailService
  ↓
NotificationService
```

```
OrderService
  ↓
PaymentService
  ↓
Logger
```

Now you may have hundreds of objects and dependencies.

Manually doing this everywhere becomes difficult:

```
UserRepository repository = new UserRepository();
EmailService emailService = new EmailService();
UserService userService = new UserService(repository, emailService);
UserController controller = new UserController(userService);
```

Spring helps manage these relationships.

4. What Is Dependency?

A **dependency** is an object that another object needs to perform its work.

Example:

```
public class UserService {

    private UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}
```

Here:

```
UserService
  ↓
```

UserRepository

UserService depends on UserRepository.

Instead of UserService creating the repository itself:

```
public class UserService {  
    private UserRepository repository = new UserRepository();  
}
```

we can provide it from outside:

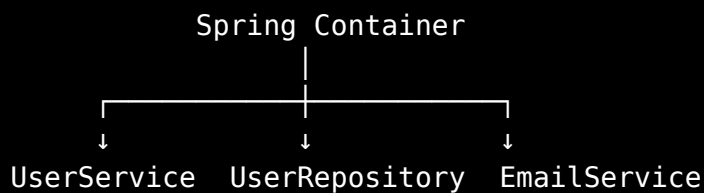
```
public class UserService {  
    private UserRepository repository;  
    public UserService(UserRepository repository) {  
        this.repository = repository;  
    }  
}
```

This idea becomes extremely important in Spring.

5. What Is the Spring Container?

The **Spring Container** is responsible for managing Spring objects.

Think of it as an object manager.



The container can:

- create objects
- store objects
- connect dependencies
- provide objects when needed
- manage their lifecycle

You don't need to understand every internal detail yet.

For now remember:

Spring Container = manages Spring beans and their dependencies.

6. What Is a Bean?

A **bean** is an object that is created and managed by the Spring container.

For example:

```
@Component
public class UserService {

    public void createUser() {
        System.out.println("User created");
    }
}
```

Because of `@Component`, Spring can discover this class and create an object of it.

Conceptually:

```
UserService class
    ↓
Spring creates object
    ↓
UserService bean
    ↓
Spring Container
```

Important distinction

A normal Java object:

```
UserService service = new UserService();
```

is not automatically a Spring bean.

A Spring-managed object:

```
@Component
public class UserService {
}
```

can become a Spring bean when component scanning is active.

7. A Complete Basic Spring Example

Before Spring Boot, understand the basic idea.

UserRepository

```
import org.springframework.stereotype.Component;

@Component
public class UserRepository {

    public void save() {
        System.out.println("User saved");
    }
}
```

UserService

```
import org.springframework.stereotype.Component;

@Component
public class UserService {
```

```
private final UserRepository userRepository;

public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
}

public void createUser() {
    userRepository.save();
    System.out.println("User created");
}
}
```

Spring sees:

```
UserRepository
    ↑
UserService
```

It creates the repository bean and provides it to the service.

8. What Is Spring Boot?

Spring Boot is a framework built on top of Spring that makes creating and running Spring applications much easier by providing sensible defaults, auto-configuration, starter dependencies, and embedded servers.

In simple terms:

```
Spring
+
Convenient setup
+
Auto-configuration
+
Starter dependencies
+
Embedded server
```


=
Spring Boot

Spring Boot does **not replace Spring**.

It makes working with Spring easier.

9. Spring vs Spring Boot

The easiest interview answer:

Spring is the core Java framework that provides features such as dependency injection and application infrastructure. Spring Boot builds on Spring and simplifies configuration, dependency setup, and application startup.

Or even shorter:

Spring provides the framework; Spring Boot makes using Spring easier.

10. Why Spring Boot Exists

Traditional Spring applications could require considerable configuration.

You might need to configure:

- dependencies
- application configuration
- web infrastructure
- server
- component scanning
- other framework settings

Spring Boot tries to provide sensible defaults.

Instead of spending lots of time configuring infrastructure:

Developer



Spring Boot

↓

Configured Spring Application

You can focus more on application logic.

11. Spring Boot Starters

One of Spring Boot's conveniences is **starter dependencies**.

For a web application, you can use:

```
org.springframework.boot  
spring-boot-starter-web
```

This gives you the dependencies commonly required for Spring web development.

You don't have to manually identify and add every underlying library.

12. Auto-Configuration

This is a **high-level concept only** for now.

Spring Boot looks at:

- dependencies you added
- application configuration
- available classes

and automatically configures many things using sensible defaults.

For example, if you add:

```
spring-boot-starter-web
```

Spring Boot recognizes that you're building a web application and configures appropriate web infrastructure.

Think:

```
Dependencies
  ↓
Spring Boot detects them
  ↓
Reasonable configuration
  ↓
Application runs
```

Interview definition

Auto-configuration is a Spring Boot feature that automatically configures parts of the application based on the dependencies and configuration present.

Don't worry about the internal implementation yet.

13. Embedded Server

Traditional Java web applications often required deploying an application to a separately installed server.

Spring Boot can package an application with an **embedded web server**.

Conceptually:

```
Spring Boot Application
  +
Embedded Server
  ↓
Run application directly
```

For example:

```
mvn spring-boot:run
```

can start the application.

You don't have to manually install and configure a separate server for basic development.

14. Spring Initializr

Spring Initializr is a tool used to generate a Spring Boot project with the required structure and dependencies.

The screenshot shows the Spring Initializr web application interface. At the top, it says "SPRING INITIALIZR bootstrap your application now". Below this, there's a form to generate a project. The form has three main sections: "Project Metadata", "Dependencies", and a "Generate Project" button.

Project Metadata

Artifact coordinates

Group:

Artifact:

Dependencies

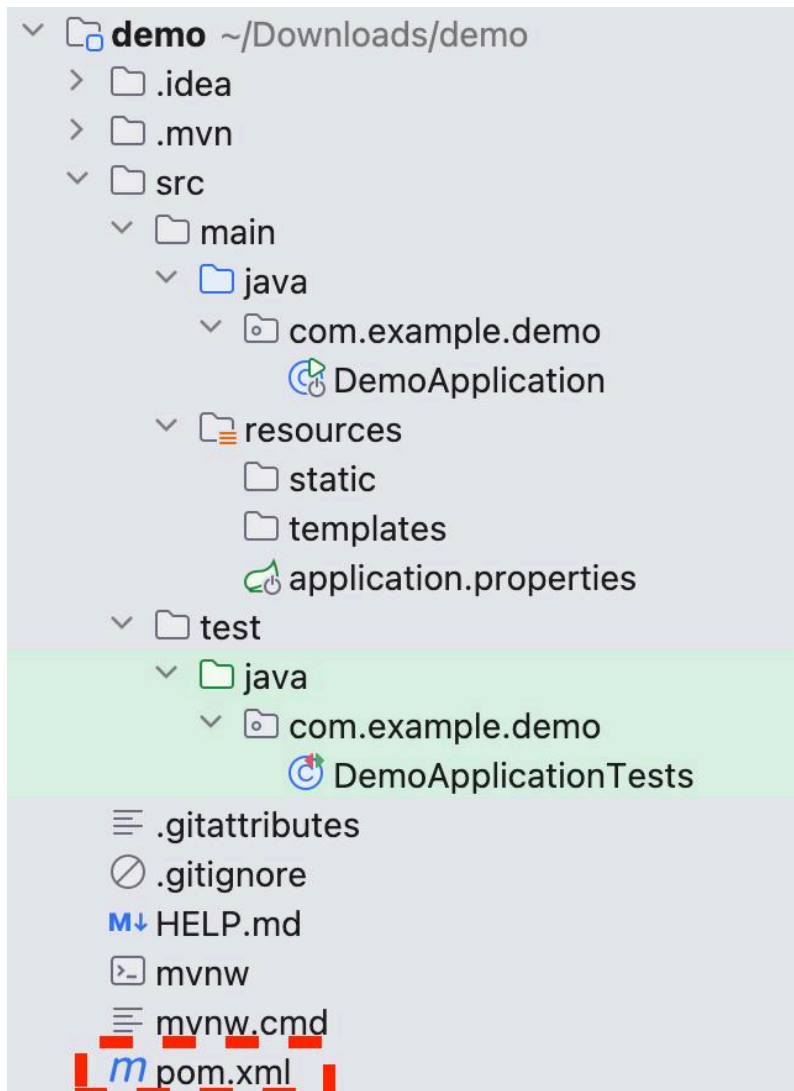
Add Spring Boot Starters and dependencies to your application

Search for dependencies:

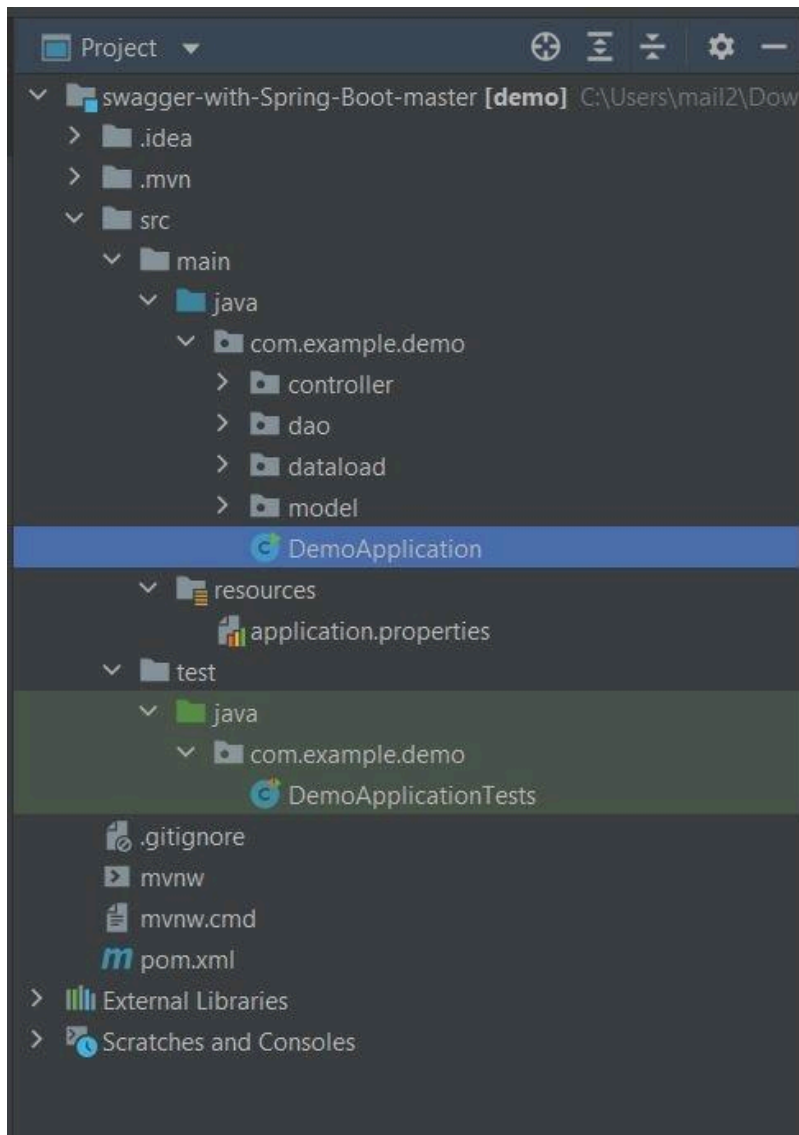
Selected Dependencies:

Don't know what to look for? Want more options? Switch to the full version.

start.spring.io is powered by [Spring Initializr](#) and [Pivotal Web Services](#).



```
my-spring-boot-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── myapp/
│   │   │   │   │   │   ├── MyAppApplication.java
│   │   │   │   │   │   ├── controller/
│   │   │   │   │   │   │   ├── ProductController.java
│   │   │   │   │   │   │   └── UserController.java
│   │   │   │   │   │   ├── service/
│   │   │   │   │   │   │   ├── ProductService.java
│   │   │   │   │   │   │   └── UserService.java
│   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   │   ├── ProductRepository.java
│   │   │   │   │   │   │   └── UserRepository.java
│   │   │   │   │   │   └── model/
│   │   │   │   │   │       ├── Product.java
│   │   │   │   │   │       └── User.java
│   │   │   │   │   └── resources/
│   │   │   │   │       ├── application.properties
│   │   │   │   │       ├── static/
│   │   │   │   │       └── templates/
│   │   │   └── resources/
│   │       ├── application.properties
│   │       ├── static/
│   │       └── templates/
│   └── resources/
│       ├── application.properties
│       ├── static/
│       └── templates/
├── mvnw
├── mvnw.cmd
├── pom.xml
└── README.md
```



You can use:

[Spring Initializr](#)

For your activity, choose:

```
Project: Maven
Language: Java
Spring Boot: Current stable version
Packaging: Jar
Java: Your installed Java version
Dependencies:
    Spring Web
```

Do **not** add:

```
JPA
Spring Security
Docker
Kafka
Spring Cloud
Microservices
```

for this exercise.

15. Generated Spring Boot Project

A typical generated project looks approximately like:

```
my-spring-app
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.example.demo
│   │   │       └── DemoApplication.java
│   │   └── resources
│   │       └── application.properties
│   └── test
│       └── java
├── pom.xml
└── mvnw
```

The exact package and filenames may differ depending on what you enter in Initializr.

16. Main Spring Boot Class

The generated class will look similar to:

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

There are **two extremely important things** here.

17. The main() Method

This is normal Java:

```
public static void main(String[] args)
```

Spring Boot does **not replace Java's main method**.

The JVM starts here.

Then:

```
SpringApplication.run(DemoApplication.class, args);
```

starts the Spring Boot application.

Conceptually:

```
JVM
↓
```

```
main()
↓
SpringApplication.run()
↓
Spring Container starts
↓
Beans/configuration initialized
↓
Embedded server starts
↓
Application running
```

18. @SpringBootApplication

The annotation:

```
@SpringBootApplication
```

is one of the most important Spring Boot annotations.

At a beginner level, understand it as:

This marks the main class of a Spring Boot application and enables important Spring Boot behavior such as component scanning and auto-configuration.

You don't need to memorize its internal annotations yet.

For interviews:

```
@SpringBootApplication
↓
Main Spring Boot application class
↓
Auto-configuration
+
Component scanning
+
Configuration
```

19. Complete Spring Boot Example

Let's build a tiny application.

Step 1 — pom.xml

A minimal Maven configuration will look similar to:

```
4.0.0
```

```
    org.springframework.boot  
    spring-boot-starter-parent  
    3.5.5
```

```
com.example  
demo  
0.0.1-SNAPSHOT
```

```
17
```

```
    org.springframework.boot  
    spring-boot-starter-web
```

```
    org.springframework.boot  
    spring-boot-starter-test  
    test
```

```
org.springframework.boot  
spring-boot-maven-plugin
```

Note: When using Spring Initializr, let it generate the current compatible Spring Boot version and Java version for your environment rather than manually copying an old version.

20. Main Class

```
package com.example.demo;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class DemoApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
}
```

21. Create a Controller

Create:

```
controller/  
HelloController.java
```

Code:

```
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Spring Boot";
    }
}
```

Now run the application.

Open:

```
http://localhost:8080/hello
```

You should get:

```
Hello Spring Boot
```

22. What Happened?

When you start:

```
SpringApplication.run(DemoApplication.class, args);
```

Spring Boot starts the application.

It discovers:

```
@RestController
public class HelloController
```

Spring manages this controller.

Then:

```
@GetMapping("/hello")
```

maps:

```
GET /hello
  ↓
HelloController.hello()
```

So when the browser requests:

```
GET /hello
```

the method executes:

```
public String hello() {
    return "Hello Spring Boot";
}
```

23. Add a Service

Now let's demonstrate the Spring container and beans.

HelloService

```
package com.example.demo.service;
```

```
import org.springframework.stereotype.Service;

@Service
public class HelloService {

    public String getMessage() {
        return "Hello from Service";
    }
}
```

Controller

```
package com.example.demo.controller;

import com.example.demo.service.HelloService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private final HelloService helloService;

    public HelloController(HelloService helloService) {
        this.helloService = helloService;
    }

    @GetMapping("/hello")
    public String hello() {
        return helloService.getMessage();
    }
}
```

Notice:

```
public HelloController(HelloService helloService)
```

We did **not** write:

```
new HelloService()
```

Spring provides the dependency.

Conceptually:

```
Spring Container
├── HelloService bean
└── HelloController bean
    └── HelloService
```

This is one of the fundamental ideas behind Spring.

24. Why Constructor Injection?

This:

```
public HelloController(HelloService helloService) {
    this.helloService = helloService;
}
```

is called **constructor dependency injection**.

It is generally preferred because:

- dependencies are explicit
- required dependencies cannot easily be omitted
- fields can be final
- classes are easier to test
- dependencies are provided from outside

For example:

```
private final HelloService helloService;
```


is preferable to hiding the dependency inside the class.

25. @Component, @Service and @RestController

At this stage, know their basic purposes.

@Component

General Spring-managed component:

```
@Component
public class MyComponent {
}
```

@Service

Typically used for business/service logic:

```
@Service
public class UserService {
}
```

@RestController

Used for REST/web controllers:

```
@RestController
public class UserController {
}
```

Don't worry yet about all the internal details.

26. Run From IDE

If you're using IntelliJ IDEA, Eclipse, or Spring Tool Suite:

Open:

```
DemoApplication.java
```

Run:

```
main()
```

Your console should show Spring Boot startup information and eventually indicate that the application is running.

Then access:

```
http://localhost:8080/hello
```

27. Run From Terminal

From the Maven project directory:

```
mvn spring-boot:run
```

Or with Maven Wrapper on Windows:

```
mvnw.cmd spring-boot:run
```

On Linux/macOS:

```
./mvnw spring-boot:run
```

You can also build the application:

```
mvn clean package
```

Then run the generated JAR:

```
java -jar target/demo-0.0.1-SNAPSHOT.jar
```

The exact JAR filename depends on your project version.

28. What Did Spring Boot Save Us From Doing?

For your Week 7 note, the important answer is:

Spring Boot saves us from manually configuring much of the application infrastructure.

Instead of manually setting up:

```
Web application infrastructure  
Server configuration  
Many dependencies  
Spring configuration  
Application startup configuration
```

Spring Boot provides:

```
Starter dependencies  
Auto-configuration  
Embedded server  
Default configuration  
Easy application startup
```

So we can focus more on:

rather than infrastructure setup.

29. Spring Boot Does NOT Replace Java

This is an important point.

Spring Boot does not replace:

- classes
- objects
- constructors
- interfaces
- inheritance
- polymorphism
- collections
- exceptions
- generics
- Java syntax
- OOP

For example, this is still ordinary Java:

```
public class User {  
  
    private String name;  
  
    public User(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Spring simply provides additional infrastructure around your Java application.

Important mindset

Don't think:

```
Spring = replacement for Java
```

Think:

```
Java
+
Spring
+
Spring Boot
=
Java application with powerful application infrastructure
```

30. Common Beginner Mistake

Wrong thinking

"I know Spring annotations, so I know Java backend."

Not necessarily.

You should understand:

```
Java
↓
OOP
↓
Collections
↓
Exceptions
↓
Generics
↓
Maven
```

```
↓  
HTTP  
↓  
SQL/JDBC  
↓  
Spring  
↓  
Spring Boot
```

Spring becomes much easier when you understand Java fundamentals.

31. Important Beginner Mental Model

Remember this flow:

```
You write Java classes  
↓  
Spring Boot starts  
↓  
Spring Container starts  
↓  
Spring discovers components  
↓  
Spring creates/manages beans  
↓  
Spring connects dependencies  
↓  
Web server starts  
↓  
HTTP requests reach controllers
```

This mental model is much more important than memorizing dozens of annotations.

32. Spring vs Spring Boot — Detailed Comparison

Spring	Spring Boot
Java framework	Built on Spring
Provides core Spring features	Simplifies Spring development
Dependency Injection	Provides convenient setup
Bean management	Auto-configuration
Application infrastructure	Starter dependencies
Can require more configuration	Convention/default based
Doesn't inherently mean embedded server	Commonly uses embedded server
More flexible/manual	More opinionated/convenient

33. Interview Points

1. What is Spring?

Answer:

Spring is a Java framework that provides infrastructure for building applications, including dependency injection, bean management, configuration, and web application support.

2. What is Spring Boot?

Answer:

Spring Boot is built on top of Spring and simplifies Spring application development through auto-configuration, starter dependencies, sensible defaults, and embedded servers.

3. Is Spring Boot a replacement for Spring?

Answer:

No. Spring Boot builds on top of Spring and makes Spring applications easier to configure and run.

4. Why was Spring created?

Answer:

Spring helps reduce application infrastructure complexity and makes object management and dependencies easier to handle.

5. What is a Spring bean?

Answer:

A Spring bean is an object that is created and managed by the Spring container.

6. What is the Spring container?

Answer:

The Spring container manages Spring beans, their creation, configuration, dependencies, and lifecycle.

7. What is dependency injection?

Answer:

Dependency injection means providing an object's required dependencies from outside instead of the object creating those dependencies itself.

Example:

```
public UserService(UserRepository repository) {  
    this.repository = repository;  
}
```

8. What is auto-configuration?

Answer:

Auto-configuration allows Spring Boot to automatically configure parts of an application based on the dependencies and configuration available.

9. What is an embedded server?

Answer:

An embedded server is a web server packaged with the application so the application can run without requiring a separately installed application server for basic use.

10. What does @SpringBootApplication do?

Answer:

It identifies the main Spring Boot application class and enables important Spring Boot functionality such as configuration, component scanning, and auto-configuration.

11. What is Spring Initializr?

Answer:

Spring Initializr is a project-generation tool that creates a Spring Boot project with the selected Java version, build system, and dependencies.

12. What is a starter dependency?

Answer:

A Spring Boot starter is a convenient dependency that brings together commonly required dependencies for a particular type of application.

Example:

34. Interview Scenario Question

Question

Suppose you have:

```
public class UserService {  
    private UserRepository repository = new UserRepository();  
}
```

What's the problem?

Answer

UserService is tightly coupled to the concrete UserRepository.

A better design is:

```
public class UserService {  
    private final UserRepository repository;  
  
    public UserService(UserRepository repository) {  
        this.repository = repository;  
    }  
}
```

Now another object can provide the repository.

Spring can manage this dependency automatically.

35. Interview Scenario: Spring vs Spring Boot

Interviewer:

What is the difference between Spring and Spring Boot?

Strong beginner answer:

Spring is a Java framework that provides features such as dependency injection and bean management. Spring Boot builds on Spring and simplifies configuration and application startup through features such as auto-configuration, starter dependencies, and embedded servers.

That's an excellent answer for your current level.

36. Interview Questions to Practice

Try answering these without looking at the answers:

Basic

1. What is a framework?
2. What is Spring?
3. Why do we need Spring?
4. What is Spring Boot?
5. Is Spring Boot different from Java?
6. Is Spring Boot a replacement for Spring?
7. What is a Spring bean?
8. What is the Spring container?
9. What is dependency injection?
10. Why is dependency injection useful?

Spring Boot

11. What is Spring Initializr?
12. What does `@SpringBootApplication` mean?
13. What does [SpringApplication.run\(\)](#) do?
14. What is auto-configuration?

15. What is an embedded server?
16. What is a Spring Boot starter?
17. Why is spring-boot-starter-web used?
18. How do you run a Spring Boot application?
19. How do you run Spring Boot from Maven?
20. How do you package a Spring Boot application?

Coding/Scenario

21. Why is new UserRepository() inside UserService less flexible?
 22. How does constructor injection solve this?
 23. What makes an object a Spring bean?
 24. What happens when a Spring Boot application starts?
 25. What happens when a request reaches /hello?
 26. Why is constructor injection commonly preferred?
 27. Does Spring eliminate the need to understand Java?
 28. What did Spring Boot save you from configuring manually?
 29. What happens if Spring cannot find a required dependency?
 30. Explain the lifecycle from main() to an HTTP request.
-

37. Your Practice Task

For your current Week 7 level, build only this:

```
Spring Boot
├── Spring Web
│   ├── HelloController
│   └── HelloService
```

Expected request

```
GET /hello
```

Expected response

```
Hello from Service
```

Your classes should be:

```
DemoApplication  
HelloController  
HelloService
```

Use:

```
@SpringBootApplication  
@RestController  
@Service  
@GetMapping
```

and **constructor injection**.

Do not add:

```
JPA  
Hibernate  
Spring Security  
JWT  
Docker  
Kafka  
Microservices  
MongoDB
```

yet.

38. Final Mental Summary

Memorize this:

```
Framework
  ↓
Reusable application structure

Spring
  ↓
Java framework
  ↓
Manages beans and dependencies

Spring Container
  ↓
Creates/manages Spring beans

Bean
  ↓
Object managed by Spring

Spring Boot
  ↓
Built on Spring
  ↓
Simplifies configuration
  ↓
Auto-configuration
  ↓
Starter dependencies
  ↓
Embedded server

Spring Boot Application
  ↓
main()
  ↓
SpringApplication.run()
  ↓
Spring Container
  ↓
Beans + dependencies
  ↓
Embedded web server
  ↓
HTTP requests
```

The one sentence you should remember for interviews:

Spring provides the application framework and dependency management, while Spring Boot makes Spring applications easier to configure, start, and run.